Assignment

AI-Assisted .NET C# Development with GitHub Copilot

GenAI Level 2 — AI-Integrated ✅

This assignment is classified as **Level 2 — AI-Integrated**. Please follow the **Level 2 GenAI requirements** described in the *Generative AI (GenAI) Use in Assessments* policy posted at the top of the Assignments section in Moodle.

Real-World Scenario: Community Sports Facility Booking System

Assignment Overview

A community recreation centre manages several facilities, including basketball courts, tennis courts, meeting rooms, and fitness studios. Currently, staff record bookings manually, making it difficult to determine facility availability, calculate booking costs, and produce useful summaries.

You have been asked to develop a **C# console application using .NET** that provides a simplified **Community Sports Facility Booking System**.

An important objective of this assignment is not only to produce a working application, but also to demonstrate an effective **AI-assisted software development workflow using GitHub Copilot**. You must use Copilot incrementally to help create, explain, review, debug, and improve parts of the application.

You remain responsible for understanding, reviewing, testing, and validating all generated code.

---

1. Learning Objectives

After completing this assignment, you should be able to:

- create and run a .NET C# project;
- design C# classes representing a real-world domain;
- work with collections of objects;
- use LINQ to filter, transform, sort, and analyze data;
- provide GitHub Copilot with useful **context, intent, clarity, and specificity**;

- use comments as prompts to guide code generation;
- refine prompts when Copilot produces an unsuitable implementation;
- review and modify AI-generated code;
- use Copilot to assist with code explanation and debugging;
- compile, run, and test a C# application;
- critically evaluate AI-generated software rather than automatically accepting it.

---

2. Required Application

Create a new .NET C# console project for the application.

For example:

```
dotnet new console -n SportsFacilityBooking
```

```
cd SportsFacilityBooking
```

Run the initial project:

```
dotnet run
```

Verify that the application compiles and executes before beginning development.

NOTE — Development Process Matters

Do not ask Copilot to generate the entire assignment in one prompt. The purpose of this assignment is to demonstrate **incremental AI-assisted development**.

---

3. Part A — Design the Domain Model

Create a Facility class containing appropriate information such as:

```
public int Id { get; set; }
```

```
public string Name { get; set; }
```

```
public string Type { get; set; }
```

```
public double HourlyRate { get; set; }
```

Create a Booking class containing appropriate information such as:

```
public int Id { get; set; }
```

```
public string CustomerName { get; set; }
```

```
public int FacilityId { get; set; }

public int DurationHours { get; set; }
```

You may add other fields if they are useful to your design.

Your program must create **at least five facilities and ten bookings** containing meaningful sample data.

---

4. Part B — Develop the Application Incrementally with Copilot

Use **comments/prompts** to progressively guide Copilot.

For example, you might begin with:

```
// Create a Facility class representing a recreation-centre facility.

// Each facility should have an ID, name, type, and hourly rental rate.
```

After reviewing the generated code, continue with another requirement:

```
// Create a Booking class representing a customer's facility booking.

// Each booking must reference a facility and contain the

// customer's name and booking duration in hours.
```

Then progressively add the application's remaining requirements.

For every significant Copilot interaction:

1. write a meaningful prompt or comment;
2. observe the generated suggestion;
3. review the suggestion;
4. determine whether it satisfies the requirement;
5. accept, partially accept, modify, or reject it;
6. refine the prompt when necessary.

Prompt Quality

Your prompts should demonstrate the four concepts discussed in class:

Context
+
Intent
+
Clarity
+

Specificity
↓
Better-Defined Request
↓
More Relevant Copilot Suggestion

At least **one example in your report must demonstrate prompt refinement** by showing an initial prompt that produced an incomplete or unsuitable result and the improved prompt used afterward.

---

5. Part C — Implement the Required Operations

Your completed application must perform all of the following operations.

1. Display Facilities

Display all available facilities, including:

- ID;
- name;
- type;
- hourly rate.

2. Display Bookings

Display all bookings with meaningful information.

Instead of displaying only FacilityId, your output should associate the booking with the corresponding facility.

3. Calculate Booking Cost

For each booking, calculate:

```
Booking Cost = Facility Hourly Rate × Duration
```

For example, if a tennis court costs $30/hour and is booked for three hours:

```
30 × 3 = $90
```

4. Filter Using LINQ

Use LINQ Where() to identify bookings lasting **two hours or more**.

The threshold should be clearly represented in your implementation.

5. Transform Data Using LINQ

Use Select() to create a result containing information such as:

```
Customer

Facility

Duration

Total Cost
```

6. Sort the Results

Use LINQ to sort bookings by **total booking cost from highest to lowest**.

7. Calculate Summary Information

Use appropriate LINQ operations to calculate and display:

- total number of bookings;
- total revenue from all bookings;
- average booking cost;
- the most expensive booking.

PITFALL — Correct Syntax Does Not Guarantee Correct Logic

A LINQ query may compile successfully while implementing the wrong business requirement. Verify what your query actually selects, filters, and calculates.

---

6. Part D — Use Copilot for Code Explanation

Select one meaningful portion of your implementation containing LINQ.

Ask GitHub Copilot to explain the code.

For example:

```
var expensiveBookings = bookings
    .Where(...)
    .Select(...)
    .OrderByDescending(...);
```

Compare Copilot's explanation with your own understanding.

Your report must briefly explain:

- what the query receives as input;
- what Where() does;
- what Select() produces;

- what the sorting operation does;
- what the final result represents.

Do not simply copy Copilot's explanation into the report.

---

7. Part E — Debugging with GitHub Copilot

Create or identify at least **one genuine programming problem** during development.

Examples include:

- an incorrect LINQ condition;
- an incorrect cost calculation;
- incorrect sorting;
- incorrect facility lookup;
- unexpected console output.

Use Copilot to investigate the problem.

Follow the debugging workflow practiced in class:

Observe Incorrect Behavior
↓
Reproduce the Problem
↓
Ask Copilot to Investigate
↓
Review Proposed Change
↓
Understand Root Cause
↓
Accept / Modify Suitable Change
↓
Run Again
↓
Verify Expected Result

Your report must document the problem, Copilot's proposed solution, and your final correction.

WARNING — Copilot Is Not the Debugger of Record

A suggestion from Copilot is not evidence that the bug has been fixed. You must execute the application and verify the result yourself.

---

8. Part F — Testing and Verification

Run the completed application using:

```
dotnet run
```

Verify that all required functionality works.

At minimum, demonstrate:

- facility data;
- booking data;
- correct facility association;
- correct booking-cost calculations;
- LINQ filtering;
- LINQ projection;
- sorting;
- total revenue;
- average booking cost;
- most expensive booking.

Test more than one case.

If Copilot produces code that does not work correctly, refine your prompt or manually correct the implementation.

---

9. Required Development Workflow

Your work should demonstrate the following overall process:

Understand Requirements
↓
Design Classes and Data
↓
Write Contextual Comments/Prompts
↓
Copilot Generates Suggestions
↓
Developer Reviews Suggestions
↓
Accept / Modify / Reject
↓
Implement LINQ Operations
↓
Compile and Run
↓

Observe Results
↓
Debug / Refine with Copilot
↓
Run Again
↓
Verify Final Application

The developer—not Copilot—is responsible for every stage of the final application.

---

10. Report and Screenshot Requirements

Submit a short but complete report documenting your practical work.

The report must contain a **comprehensive set of screenshots** demonstrating that you personally completed the required activities.

Your screenshots should clearly show:

1. creation and structure of the .NET project;
2. the C# model classes;
3. sample facility and booking data;
4. meaningful comments/prompts given to GitHub Copilot;
5. Copilot-generated suggestions;
6. at least one example of **prompt refinement**;
7. your LINQ Where() implementation;
8. your LINQ Select() implementation;
9. sorting and summary calculations;
10. Copilot being used to explain code;
11. the debugging process and proposed correction;
12. the final source code;
13. successful execution using dotnet run;
14. final console output demonstrating all major requirements.

Screenshots should be readable and organized in the same sequence as the development process. Include a short caption or explanation for each major screenshot.

WARNING — Screenshots Are a Required Part of the Assignment

Showing only the final source code and final output is **not sufficient**. Your screenshots must demonstrate the development process, your interactions with GitHub Copilot, your review of generated code, and successful execution of the application.

---

11. What to Submit to Moodle

Submit the following through **Moodle**:

- your complete C#/.NET project or source files as instructed;
- your report;
- the comprehensive set of screenshots documenting the practical work.

All submitted files should be clearly named and organized.

You must respect the due date specified on Moodle.

---

12. Mandatory Presentation to the Professor

In addition to submitting the assignment on Moodle, **you must present and demonstrate your completed assignment to the professor**.

During the presentation, you may be asked to:

- run the application;
- explain your classes;
- explain a LINQ query;
- explain one of your Copilot prompts;
- describe why you accepted or modified a Copilot suggestion;
- explain the debugging process;
- make a small modification to the program;
- explain part of your code.

IMPORTANT — GRADING REQUIREMENT

Submission on Moodle alone is not sufficient. The assignment must also be presented to the professor in order to receive a grade. An assignment that is submitted but not presented will not be graded.

---

13. Assessment Rubric

Criterion	Weight	Expectations
C#/.NET Application	30%	Correct project structure, classes, collections, calculations, and working application

LINQ Implementation	20%	Correct and meaningful use of filtering, projection, sorting, and aggregate operations
GitHub Copilot Workflow	20%	Effective prompts/comments, appropriate context, prompt refinement, review and modification of generated code
Testing & Debugging	15%	Application executed and verified; debugging process demonstrated using Copilot appropriately
Report & Screenshots	15%	Clear report with comprehensive screenshots documenting all major practical requirements
Total	**100%**	

Final Reminder

This assignment evaluates both the **final C# application** and the **development process used to create it**.

The expected workflow is:

Requirement → Context/Comments → Copilot Suggestion → Developer Review → Implementation → LINQ Processing → Run → Test → Debug/Refine → Verify

Do not treat GitHub Copilot as a mechanism for obtaining a finished assignment. Use it as an **AI pair-programming assistant**, while maintaining responsibility for the application's architecture, code correctness, testing, and final results.